

Alberta Buck – Worked Examples

Perry Kundert

2026-05-09

1 Vehicle Insurance Lifecycle

This example traces a single year in the life of a vehicle-insurance BUCK_CREDIT: from collateral creation through AMM liquidity provision and swap-fee collection to final burn. Every figure in this section is generated from an actual Foundry test run – see `test/BuckLifecycle.t.sol` for the on-chain numbers and `alberta_buck/test/test_lifecycle_plot.py` for the visualisation.

1.1 Generating the data

```
forge test --match-test test_lifecycle -vv
```

This writes `test/vectors/lifecycle.json`. Run once; re-run whenever the contracts change. To regenerate the plot:

```
python -m pytest alberta_buck/test/test_lifecycle_plot.py -v -s
```

1.2 Scenario

Alice owns a \$10,000 vehicle. Her insurer issues a BUCK_CREDIT NFT with a 15 %/yr declining-balance depreciation schedule and a \$500 salvage floor (5 %). Alice activates the full face value and mints 5,000 BUCK against it.

Parameter	Value
Face value	\$10,000
Depreciation type	DECLINING_BALANCE
Annual rate	15 % (1500 bp)
Salvage floor	\$500 (5 %)
Annual premium rate	1.5 % (150 bp)
BUCK minted	5,000 BUCK
Pool seed	5,000 BUCK + 5,000 USDC
Bob's monthly swaps	$12 \times 500 \text{ USDC} \rightarrow \text{BUCK}$
Pool fee	0.3 % (Uniswap V2)

The mint draws cheapest-first coverage. At a 1.5 % premium and $10\times$ pool-ROI inverse, the per-BUCK denom is

$$\text{denom} = 10000 - 150 \times 10 = 8500$$

so `take = 5000 × 10000 / 8500 = 5882 BUCK` of coverage is drawn and `poolPrincipal = 882 BUCK` is minted to `insurancePool`. Total supply after mint: \$5882.

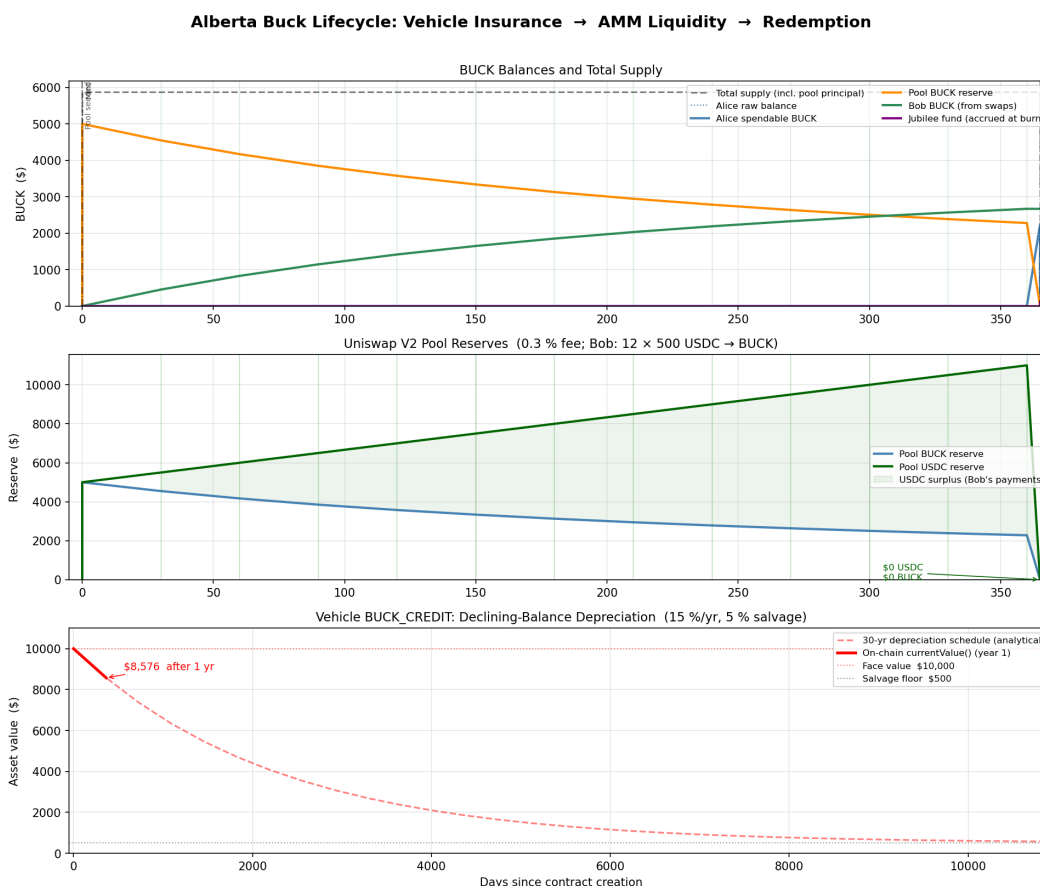
Alice deposits all 5,000 BUCK plus 5,000 USDC into a Uniswap V2 pair (1:1 ratio). Bob then buys 500 USDC of BUCK each month for a year.

1.3 Key outcomes after one year

Event	BUCK balance	USDC balance	Notes
Mint	5,000	10,000	882 BUCK principal to pool
Pool seed	0	5,000	All BUCK deposited
After swap 12	0	5,000	5000 USDC→BUCK paid in by Bob
Liquidity removed	2,232	16,000	Pool rebalanced; USDC up \$6K
After burn	0	16,000	Coverage unwound; jubilee \$117

Alice exits with \$16,000 USDC – a \$6,000 gain from Bob’s swap fees – having paid zero demurrage herself (her BUCK sat in a Carrying pool the whole time, pre-accruing Jubilee instead). The vehicle credit shows \$8,576 current value after one year of 15 % declining-balance depreciation.

1.4 Lifecycle chart



The chart has three panels:

- **BUCK balances (top):** Alice’s raw and spendable BUCK track identically until liquidity removal, when the pool’s carried `buckSeconds` arrive with the returned tokens (raw > spendable by the \$46 carried fee). Bob’s BUCK accumulates linearly as he buys each month. The total supply stays flat at \$5,882 until Alice burns, whereupon it drops to \$3,256 (the pool principal stays because `insurancePool` was only partially wound back against Alice’s partial burn). The Jubilee spike at burn day reflects the 2 %/yr accrual on the full supply over the year.

- **Pool reserves (middle):** Uniswap V2’s constant-product invariant ($K = 5000 \times 5000 = 25M$) keeps the pool balanced. Each 500 USDC swap shifts roughly 453 BUCK out of the pool. By month 12 the pool holds ~\$2,278 BUCK and ~\$11,000 USDC. Alice’s LP share captures all of this: she added \$5,000 of each, exits with \$2,232 BUCK and \$11,000 USDC.
- **Credit depreciation (bottom):** The on-chain `currentValue()` trace (thick red) follows the year-1 segment of the full 30-year schedule (dashed). The discrete whole-year compounding formula produces a linear interpolation within each year, converging to \$8,576 at day 365. After 30 years the credit would reach its \$500 salvage floor.

1.5 Demurrage and Jubilee mechanics

During the year Alice holds no BUCK directly – it lives in the Uniswap V2 pair, which is bound as a Carrying account. Carrying accounts do not lose visible balance to demurrage; instead `_accrueJubilee` credits the Jubilee fund at the full `totalSupply` rate on every mint/burn.

When Alice removes liquidity, the pair executes a `_carryingTransfer` to her. The proportional `buckSeconds` for the BUCK she receives:

$$\text{carried} = \text{liveBs} \times \text{value} / \text{raw}$$

arrive alongside the raw BUCK, so her spendable is \$2,232 while her raw balance is \$2,278 (the \$46 difference is the carried fee that locks into her slot).

The Jubilee accrues visibly only at the final `burn` call: $\$5,882 \times 2 \% \times 1 \text{ yr} = \117 . Governance can direct this accumulation to default-coverage, ground-rent disbursement, or a periodic balance reset – all on-chain paths land on the Jubilee slot.

2 Reproducing the Example

```
# From the repo root inside nix develop:
forge test --match-test test_lifecycle           # generates test/vectors/lifecycle.json
python -m pytest alberta_buck/test/test_lifecycle_plot.py -v -s # regenerates images/lifecycle.
```

The lifecycle JSON is committed alongside the test so the plot can be regenerated without re-running the Forge suite. CI regenerates it and checks for drift with `git diff --exit-code`, the same discipline used for the SNARK test vectors.